

202044502
Programming with
JAVA

Unit -3
(Part -2)
Topic – Exception Handling

Academic Year: 2024-25 (II)

Prepared By:
Prof. Pratik B. Soni
(Asst. Prof. – I.T. Dept, MBIT)

1

Exception Handling

- Basics
- try - Catch
- Nested try
- “throw” and “throws” Statement
- “finally” statement
- Creating own Exception class

2

Exception Handling- Basics

In java exception is an object which can be occurred due to piece of code of any program. When any exception is occurred then the object is created and it will be handled.

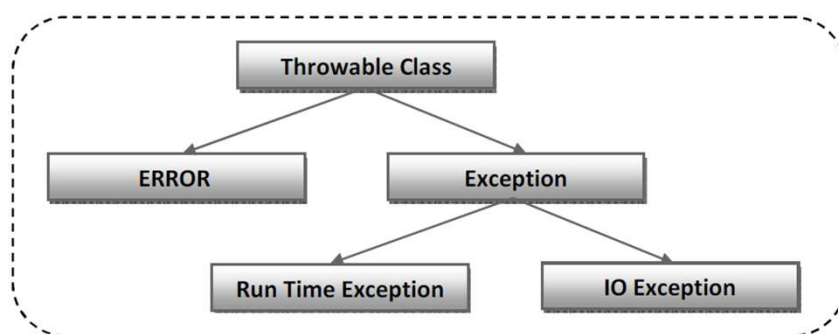
Exception can be defined as it is an error which can be occurred at runtime. And it can't be fetch at compile time. Generally exception is generated due to some reasons;

- e.g.)
- 1) A user has entered invalid data.
 - 2) A file that needs to be opened cannot be found.

3

Exception Handling- Basics

Exception Hierarchy;



In Exception handling, there are with five different keywords mainly used.

- 1) try
- 2) catch
- 3) throw
- 4) throws
- 5) finally

4

Exception Handling- Basics

- **Unchecked exception:**

In java, "RuntimeException" class is available which is used as a super class to handle the errors at runtime. Here no explicit code is available in the program to handle them. All these exceptions under this class called "unchecked exception".

- **Checked exception:**

In this the exception is handled in the program with the try – catch mechanism. So, when the exception will be generated at run time then it will be handled by try – catch block of the program and these exceptions are called "checked Exception"

5

Exception Handling- try catch

Use of "try" "catch" block:

Syntax:

```
try
{
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb)
{
    // exception handler for ExceptionType1
}
```

6

Exception Handling- try catch

```
class data
{
    public static void main(String args[])
    {
        int d, a;
        try
        {
            d = 0;
            a = 42 / d;
            System.out.println("This will not be printed.");
        }
        catch (ArithmeticException e)
        {
            System.out.println("Division by zero.");
        }

        System.out.println("After catch statement.");
    }
}
```

7

Exception Handling- try catch – Multiple catch

```
try
{
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb) -
{
    // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb)
{
    // exception handler for ExceptionType1
}
.....
.....
catch (ExceptionType-n exOb)
{
    // exception handler for ExceptionType1
}
```

8

Exception Handling- try catch – Multiple catch

```
class MultiCatch
{
    public static void main(String args[])
    {
        try
        {
            int a = 0;
            System.out.println("a = " + a);
            int b = 42 / a;
            int c[] = { 1 };
            c[42] = 99;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Divide by 0: " + e);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Array index oob: " + e);
        }

        System.out.println("After try/catch blocks.");
    }
}
```

9

Exception Handling- try catch – Nested try

```
try
{
    int a = Integer.parseInt(args[0]);    // using command line args
    int b = 42 / a;
    System.out.println("a = " + a);

    try
    {
        if(a==1)
        {
            a = a/(a-a);
        }
        if(a==2)
        {
            int c[] = { 1 };    // array - c with only one value
            c[42] = 99;
        }
    }
    catch(ArrayIndexOutOfBoundsException e)
    {
        System.out.println("Array index out-of-bounds: " + e);
    }
}
catch(ArithmeticException e)
{
    System.out.println("Divide by 0: " + e);
}
```

10

Exception Handling- “throw” & “throws”

- If we want to throw the exception explicitly from the program the “throw” statement is used.
- *Syntax:*
 - **throw <exception object>;**
- when any “throw” statement is executed then it is directly going for exception object class’s constructor. Exception class constructor contains either one argument or no arguments.
- when any exception is thrown then it is directly find out the next “catch” statement from the same function or class.
- If, any “catch” statement is not found in the function or any class then it will be finding out “catch” statement from where the function is called.

11

Exception Handling- “throw” & “throws”

- If a method is capable of causing an exception that it does not handle, it must specify this behaviour so that callers/users of the method can guard themselves against that exception.
- In such condition “throws” can be used with the function / class declaration.
- *Syntax:*
 - **class <name> throws <exception - list>**

```
{
}
```
 - **<ret-type> <funname> (<param-list>) throws <excep-list>**

```
{
}
```

12

Exception Handling- “throw” vs “throws”

throw	throws
throw keyword is used to throw an exception explicitly.	throws keyword is used to declare an exception possible during its execution.
throw keyword is followed by an instance of Throwable class or one of its sub-classes.	throws keyword is followed by one or more Exception class names separated by commas.
throw keyword is declared inside a method body.	throws keyword is used with method signature (method declaration).
We cannot throw multiple exceptions using throw keyword.	We can declare multiple exceptions (separated by commas) using throws keyword.

13

Exception Handling- “throw” & “throws”

```

class Test
{
    static void check() throws ArithmeticException
    {
        System.out.println("Inside check function");
        throw new ArithmeticException("demo");
    }
    public static void main(String args[])
    {
        try{
            check();
        }
        catch(ArithmeticException e)
        {
            System.out.println("caught" + e);
        }
    }
}

```

14

Exception Handling- “finally”

- “finally” block is used in JAVA.
- *Syntax:*
 - **finally**

```
{
}
```
- “finally” block must be executed after try/catch is executed.
- It means that if without any exception the try block is executed then “finally” will be executed. If any exception occurs in try block then catch block is executed and after catch block “finally” block is executed.
- Generally “finally” block will be used after catch statements.

15

Exception Handling- “finally”

```
Class ExceptionTest
{
    public static void main(String[] args)
    {
        int a[] = new int[2];
        System.out.println("out of try");
        try{
            System.out.println("Access invalid element"+ a[3]);
        }
        catch(ArrayIndexOutOfBoundsException ae) {
            System.out.println(ae);
        }
        finally
        {
            System.out.println("finally is always executed.");
        }
    }
}
```

16

Exception Handling- “finally”

- “finally” block is used in JAVA.
- *Syntax:*
 - **finally**

```
{
}
```
- “finally” block must be executed after try/catch is executed.
- It means that if without any exception the try block is executed then “finally” will be executed. If any exception occurs in try block then catch block is executed and after catch block “finally” block is executed.
- Generally “finally” block will be used after catch statements.

17

Exception Handling- “creating own Exception”

- Steps:
 - (1) Create class which *extends* “Exception” class.
 - (2) Provide one “private” variable to it.
 - (3) Provide one “constructor” to it.
 - (4) Provide “toString()” method to print exception Information.
- To “throw” user define exception from any program then “throw” statement is required.
- Next Program defines the user created exception class for;
 - If the number is grater than 10 (no >10) then exception is generated.
 - And exception is caught in the user define exception class.

18

Exception Handling- “creating own Exception”

- **PROGRAM**

Write a Java Program which has Compute Method which take one Integer as an argument. If Integer value is upto 10 then it prints the value. If not then it generates an exception for more than 10 exception. Do provide all classes and methods to fulfil above requirements.

19

Exception Handling- “creating own Exception”

```
class MyException extends Exception
{
    private int detail;
    MyException(int a)
    {
        detail = a;
    }
    public String toString()
    {
        return "MyException[" + detail + "];"
    }
}
```

```
class ExceptionDemo
{
    static void compute(int a) throws MyException
    {
        System.out.println("Called compute(" + a + ")");
        if(a > 10)
        {
            throw new MyException(a);
        }
        System.out.println("Normal exit");
    }

    public static void main(String args[])
    {
        try
        {
            compute(1);
            compute(20);
        }
        catch (MyException e)
        {
            System.out.println("Caught " + e);
        }
    }
}
```

20